



Smart Home

Vytvořte aplikaci pro virtuální simulaci inteligentního domu, skládajícího se z chytrých zařízení, umístěných v různých místnostech. Ta jsou schopna monitorovat okolní prostředí a sledovat svůj stav, spotřebu apod. Se všemi zařízeními je možné interagovat jak systémově, tak uživatelsky (jednotlivými členy domácnosti).

Součástí aplikace je i předpřipravená konfigurace domu, na které jsou ukázány implementované požadavky.



Funkční požadavky

Povinné

1. Aplikace obsahuje alespoň 8 typů spotřebičů, jako např. televize, senzor, myčka, chytré okno apod. Každé zařízení je umístěno v konkrétním pokoji, nacházejícím se v jednom patře domu (případně v garáži, na zahradě apod.).
2. Interakce se zařízeními probíhá přes **API**; primárně interakcí uživatele (člena domácnosti) – *zapnu televizi, naplním lednici* apod. Základní funkce lze ovládat i systémově – např. *blíží se bouřka - vypnu všechna el. zařízení*.
3. Interakcí se zařízením dochází ke změně jeho stavu, který ovlivňuje jak s ním můžeme interagovat dále – např. *nemohu si uvařit oběd, pokud je vypnutý sporák*. Některá zařízení mohou mít obsah – např. *lednice má jídlo, CD přehrávač má CD* apod.
4. Každý člen domácnosti má předem definované, jaká zařízení a jak s nimi může interagovat; *kočka nemůže zapnout myčku, jen tatínek opraví televizi* apod.
5. Rodina je aktivní a volný čas tráví zhruba v poměru 50 % používání spotřebičů v domě a 50 % sportem (kolo, lyže). Pokud není volné zařízení nebo sportovní náčiní, osoba čeká.
6. Každé zařízení má spotřebu (elektřiny, vody, plynu apod.); spotřeba se dá zjistit pomocí vystaveného **API**.
7. Zařízení a osoby se v každém okamžiku nacházejí v jedné místnosti, a náhodně generují události (eventy), které mohou být důležitou informací nebo upozorněním (alertem).
8. Události jsou odbavovány vhodnou osobou nebo zařízením. Například:
 - Čidlo na vítr → vytažení venkovních žaluzií
 - Jistič (výpadek elektřiny) → vypnutí všech nedůležitých spotřebičů (v provozu zůstávají pouze ty nutné)
 - Čidlo na vlhkost (prasklá trubka) → máma: zavolání hasičů, táta: uzavření vody, dcera: vylovení křečka
 - Miminko potřebuje přebalit → táta se skrývá, máma: přebalení
 - Zařízení přestalo fungovat → ...
 - V lednici došlo jídlo → ...

9. Aplikace je schopna na konci běhu vygenerovat následující reporty:

- **HouseConfigurationReport** obsahuje veškerá konfigurační data domu v hierarchii (např. dům → patro → místnost → okno → žaluzie), včetně členů domácnosti.
- **EventReport** obsahuje seznam událostí (viz výše), seskupené podle typu, zdroje a cílové entity (kdo událost odbavil).
- **ActivityAndUsageReport** obsahuje informace o tom, kolikrát která osoba použila které zařízení.
- **ConsumptionReport** informuje o spotřebě (vody, plynu, elektřiny apod.) jednotlivých zařízení.

10. Pro aplikaci je předpřipravená jedna názorná konfigurace domu, která obsahuje minimálně:

- 6 členů domácnosti; každý může mít v domě jiná oprávnění
- 20 kusů spotřebičů; alespoň 10 různých druhů
- 5 kusů sportovního náčiní
- 6 místností + venkovní prostor (zahrada, dvorek)

Bonusové

1. Spotřebiče mají životnost, která s časem klesá; zařízení se tedy může rozbít. Vybraní členové domácnosti mohou zařízení opravit; oprava má vyšší prioritu než volnočasové aktivity. Dokud není zařízení opraveno, nemůže být používáno členy domácnosti a nereaguje na systémové ovládání.
2. Pokud domácnost zjistí, že je některých zařízení nedostatek, může si do domu pořídit nové.



Nefunkční požadavky

1. Není požadována autentizace ani autorizace.
2. Aplikace může běžet pouze v jedné JVM.
3. Metody a proměnné, které nemají být dostupné ostatním třídám, mají být dobře skryté. Vygenerovaný Javadoc by měl obsahovat co nejméně public metod a proměnných.
4. Reporty jsou generovány do textového souboru.
5. Konfigurace domu, zařízení a obyvatel může být načítána přímo z třídy nebo externího souboru (preferován je JSON).



Vhodné design patterny

- State machine
- Iterator
- Factory / Factory method
- Decorator
- Singleton
- Visitor
- Observer
- Chain of Responsibility
- Partially persistent data structure
- Object Pool
- Lazy Initialization

- Monáda (2 body místo jednoho za implementaci vlastní monády)
- Streamy

Doporučená literatura

- <https://www.ikea.com/cz/en/cat/smart-home-hs001/>^[1]
- <https://www.alza.cz/zigbee>^[2]

Hypertext Links

[1] <https://www.ikea.com/cz/en/cat/smart-home-hs001/>

[2] <https://www.alza.cz/zigbee>

courses/b6b36omo/sem/smart_home.txt · Last modified: 2025/09/20 21:08 by jarymiro

Copyright © 2025 CTU in Prague | Operated by [IT Center of Faculty of Electrical Engineering](#) | Bug reports and suggestions [ServiceDesk CTU](#)